



**COLÉGIO PEDRO II – SÃO
CRISTÓVÃO III**

**CURSO TÉCNICO –
DESENVOLVIMENTO DE
SISTEMAS**

Gusthavo Paulo Sobral Costa de Souza

GSOBRAL PETHUB: PLATAFORMA WEB PARA CONEXÃO DE PRESTADORES DE SERVIÇOS E PRODUTOS DO MERCADO PET

Trabalho de Conclusão de Curso apresentado ao Colégio Pedro II, como requisito parcial para obtenção do título de Técnico em Desenvolvimento de Sistemas.

RIO DE JANEIRO - 2025

PROFESSOR ORIENTADOR: FLÁVIO COSTA

COORDENADOR: FLÁVIO COSTA

Rio de Janeiro – RJ

2025

Gusthavo Paulo Sobral Costa de Souza

GSOBRALPETHUB

Trabalho de Conclusão de Curso apresentado ao Curso Técnico em Técnico em Desenvolvimento em Sistemas do Colégio Pedro II, orientado pelo Prof. Flavio Costa e João Lagôas, como requisito parcial para obtenção do título de técnico em Desenvolvimento de Sistemas.

Rio de Janeiro- RJ 2025

AGRADECIMENTOS:

Gostaria de agradecer por todos os professores que me ajudaram e tiveram paciência comigo nesse caminho, sou extremamente grato por ter tido a oportunidade de aprender as milhares de coisas que aprendi com todos vocês!

“Tornou-se chocantemente óbvio que a nossa tecnologia excedeu a nossa humanidade”.

ALBERT EINSTEIN

RESUMO

Este trabalho apresenta o desenvolvimento do **GSobral PetHub**, uma plataforma web de natureza *marketplace* destinada a conectar tutores de animais de estimação a prestadores de serviços e produtos especializados no dinâmico e crescente segmento pet brasileiro.

O sistema foi integralmente desenvolvido utilizando um **stack tecnológico moderno e desacoplado**, englobando **HTML5, CSS3, JavaScript (ES6+)** para a camada de *frontend*, e o **Supabase** como *Backend as a Service* (BaaS) para o gerenciamento de banco de dados **PostgreSQL** e autenticação de usuários.

A arquitetura implementada oferece funcionalidades robustas, como o cadastro de perfis diferenciados, mecanismos avançados de listagem e filtro, um módulo completo de e-commerce com carrinho de compras persistente e o envio automático de notificações por e-mail via **EmailJS**. A escolha da arquitetura BaaS é estratégica para o modelo de **Produto Mínimo Viável (MVP)**, posicionando o projeto como um potencial **Startup** devido ao seu baixo custo operacional inicial e alta escalabilidade. Os resultados da implementação demonstram a plena aderência aos requisitos funcionais e não-funcionais estabelecidos, comprovando a viabilidade técnica da solução proposta para resolver o problema de fragmentação da oferta de serviços no mercado pet.

Palavras-chave: Plataforma Web. Marketplace. Startup. PostgreSQL. MVP. Mercado Pet.

1 INTRODUÇÃO

1.1 Contextualização e Justificativa

O mercado de produtos e serviços para animais de estimação no Brasil tem demonstrado uma trajetória de crescimento ininterrupto e resiliência econômica, posicionando o país entre os maiores mercados globais no setor. Essa expansão, no entanto, é acompanhada por uma notável **fragmentação da oferta de serviços**, onde tutores frequentemente enfrentam dificuldades em localizar, comparar e contratar profissionais especializados (como *pet sitters* ou adestradores) e pequenos comércios (Pet Shops locais) em uma única plataforma confiável. A ausência de um **hub digital centralizador** que promova a transparência e a eficiência na conexão direta entre a demanda do tutor e a oferta do prestador justifica o desenvolvimento do GSobral PetHub.

O crescente e robusto mercado *pet* brasileiro, embora vasto, apresenta notáveis **ineficiências e lacunas sociais** que dificultam a formalização de profissionais autônomos e a visibilidade de iniciativas não-lucrativas. O problema central que esta plataforma busca resolver reside na **extrema dificuldade enfrentada por *pet sitters* e cuidadores independentes** para encontrar um canal de trabalho que seja **não-burocrático** e que ofereça promoção direta de seus serviços. Tais profissionais frequentemente dependem de redes sociais ou indicações informais, limitando significativamente o seu alcance e potencial de crescimento.

Nesse contexto, a **disponibilidade de fácil acesso a uma plataforma de emprego e promoção** torna-se um fator de **inclusão econômica** vital para a categoria de *pet sitters*, permitindo que eles transformem um serviço informal em uma fonte de renda estável e visível. Adicionalmente, o projeto reconhece a **crítica falta de visibilidade enfrentada por Organizações Não Governamentais (ONGs)** de proteção animal. Essas entidades, que dependem diretamente de doações e adoções, raramente possuem um espaço digital centralizado para alcançar o público, perpetuando o problema do abandono e da escassez de recursos.

Finalmente, a plataforma também tangencia a complexa questão dos **baixos salários na medicina veterinária** ao fornecer um **canal de renda complementar**. Ao permitir que veterinários cadastrem serviços especializados e consultas avulsas em um *marketplace*, a plataforma oferece uma via para **diversificação de faturamento**, mitigando as pressões econômicas do setor e valorizando a expertise profissional. O GSobral PetHub, portanto, justifica sua existência como um hub digital que não apenas conecta, mas também **empodera economicamente** as pontas mais vulneráveis e menos visíveis do ecossistema *pet*.

1.2 O Público-Alvo: Um Marketplace de Dois Lados

O GSobral PetHub é projetado para servir a um **público-alvo duplo e complementar**, essencial para o funcionamento de qualquer *marketplace*:

1. **Lado da Demanda (Tutores/Clientes):** Indivíduos que possuem animais de estimação e necessitam de serviços especializados (consultas, cuidados, passeios) ou produtos específicos (ração, acessórios, medicamentos). Eles buscam conveniência, variedade e, futuramente, **prova social (avaliações)** para tomar decisões informadas.
2. **Lado da Oferta (Prestadores/Vendedores):** Profissionais e estabelecimentos que fornecem serviços e produtos. Esta categoria é segmentada no sistema em: **Veterinários, Pet Shops, Pet Sitters/Cuidadores, Adestradores, ONGs e Doadores**. Eles buscam **visibilidade**, uma ferramenta digital de fácil uso para cadastrar seus serviços/produtos e um canal direto para receber pedidos (RF06).

O sucesso do projeto depende da atração e retenção de ambos os lados, sendo o principal desafio técnico a criação de uma experiência de usuário (UX) que atenda às necessidades distintas de cada grupo.

1.3 O Projeto como Produto Mínimo Viável (MVP) e Potencial Startup

O GSobral PetHub, em sua fase de TCC, é concebido como um **Produto Mínimo Viável (MVP)**. Este raciocínio implica que a aplicação possui funcionalidades suficientes (carrinho, listagem, notificação) para **testar a hipótese de mercado** (a necessidade de um hub centralizado), mas ainda carece de funcionalidades avançadas de comercialização (como o *gateway* de pagamento).

A escolha do *stack* tecnológico (BaaS/Supabase) é tipicamente adotada por **Startups** devido a:

- **Velocidade de Desenvolvimento:** Redução drástica do tempo de codificação de *backend*.
- **Baixo Custo Inicial:** O modelo BaaS geralmente oferece planos gratuitos ou de baixo custo para o início da operação.
- **Escalabilidade:** A transição para a versão paga do serviço permite que o sistema cresça em usuários e dados sem exigir uma refatoração completa da infraestrutura.

Portanto, o GSobral PetHub não é apenas um projeto acadêmico, mas o **protótipo funcional** de uma potencial startup no nicho de tecnologia para o mercado pet.

2 REFERENCIAL TEÓRICO

2.1 E-commerce e Arquitetura de Marketplace

O modelo de *marketplace* exige uma arquitetura de dados capaz de lidar com transações entre múltiplos compradores e múltiplos vendedores, diferente do e-commerce tradicional de um único fornecedor. A complexidade do **RF09 (Agrupamento por Vendedor)** é um reflexo direto dessa arquitetura. O GSobral PetHub se apoia em sistemas de *backend* robustos para garantir a **integridade transacional**, essencial para que o modelo de negócios seja sustentável.

2.2 Arquiteturas Modernas: BaaS e Full-Stack

A arquitetura de *Backend as a Service* (BaaS) representa uma evolução da forma como aplicações *full-stack* são construídas. O BaaS (Supabase) permite delegar a gestão de infraestrutura, autenticação e APIs, liberando a equipe para focar na camada de **Lógica de Negócios e Experiência do Usuário (UX)**, o que é um fator decisivo para o sucesso e a agilidade de um MVP.

3 METODOLOGIA DE DESENVOLVIMENTO

3.1 Abordagem e Ferramentas

O projeto seguiu a **metodologia ágil**, com um ciclo de desenvolvimento dividido em *Sprints* focadas em funcionalidades específicas. O desenvolvimento utilizou o **VS Code** e o navegador Google Chrome para testes. O foco da metodologia foi a entrega contínua de valor, priorizando a funcionalidade principal (*marketplace*).

3.2 Requisitos do Sistema

Categoria	Código	Descrição do Requisito
Funcional	RF05	O sistema deve permitir a listagem e filtragem de prestadores e produtos por categoria e localização.
Funcional	RF07	O sistema deve ter um Módulo de Carrinho que persista os produtos selecionados no banco de dados.
Funcional	RF09	O sistema deve processar o <i>checkout</i> e agrupar os itens por vendedor para notificação correta.
Funcional	RF10	O sistema deve enviar notificações por e-mail ao vendedor após a finalização da compra.
Não-Funcional	RNF01	O <i>layout</i> deve ser responsivo e adaptável a <i>smartphones</i> (Mobile-First).
Não-Funcional	RNF04	O design deve ser minimalista e a navegação, intuitiva (UX).

Categoria	Código	Descrição do Requisito
Não-Funcional	RNF05	O sistema deve garantir a segurança de senhas e dados de usuário (hashing e HTTPS).

4 DESENVOLVIMENTO E IMPLEMENTAÇÃO TÉCNICA

O desenvolvimento do GSobral PetHub é a materialização do raciocínio arquitetural de um *stack* moderno. Este capítulo detalha a função de cada componente tecnológico, conforme solicitado.

4.1 Fundamentos da Interface e Estilização

4.1.1 HTML (*HyperText Markup Language*)

O **HTML5** foi utilizado para fornecer uma **estrutura semântica** robusta e acessível. A separação estrita entre marcação e lógica (*views* vs. *scripts*) foi mantida em arquivos como `perfil-prestador.html`, garantindo a clareza do código. O uso de elementos semânticos é vital para a otimização em motores de busca, uma preocupação estratégica para um futuro modelo de negócio.

```

C:\Users > PC > Desktop > gsobralpethub > frontend > gsobralpethubpaginainicial.html > {} "gsobralpethubpaginainicial.html" > html
1  <!DOCTYPE html>
2  <html lang="pt-br">
9  <body>
43 <main class="main-content">
117 </section>
118 </main>
119
120 <!-- Modal de Perfil -->
121 <div class="modal-overlay hidden" id="modalOverlay">
122   <div class="modal">
123     <button class="modal-close" onclick="toggleModal()">&times;</button>
124     <h3 class="modal-title">Complete seu Perfil</h3>
125
126     <div class="photo-upload">
127       <div class="photo-circle" id="photoCircle" onclick="document.getElementById('photoInput').click()">
128         <span class="photo-placeholder">📷</span>
129       </div>
130       <span class="photo-label">Adicionar Foto</span>
131       <input type="file" id="photoInput" accept="image/*" style="display: none;" onchange="handlePhotoUpload(event)">
132     </div>
133
134     <form id="userForm">
135       <div class="form-group">
136         <label class="form-label">Nome de Usuário</label>
137         <input type="text" class="form-input" id="username" required placeholder="Nome de Usuário">
138       </div>
139
140       <div class="form-group">
141         <label class="form-label">Descrição</label>
142         <textarea class="form-textarea" id="description" required placeholder="Conte um pouco sobre você...">
143       </div>
144

```

Código HTML pertencente à página principal do site

4.1.2 CSS (Cascading Style Sheets)

O **CSS3** foi aplicado de forma a garantir o **RNF01 (Responsividade)** através da abordagem **Mobile-First**. As regras de estilo em `perfil-prestador.css` e `carrinho.css` utilizam flexbox e CSS Grid com *Media Queries* para adaptar o *layout* de forma fluida. O design minimalista (RNF04) é mantido pela paleta de cores escura (#0a0a0a) e pelo uso da fonte *Montserrat*, que confere profissionalismo e legibilidade à interface.

```

1  * {
2      margin: 0;
3      padding: 0;
4      box-sizing: border-box;
5  }
6
7  body {
8      font-family: 'Montserrat', sans-serif;
9      background-color: #0a0a0a;
10     color: #fff;
11     overflow-x: hidden;
12 }
13
14 .header {
15     position: fixed;
16     top: 0;
17     left: 0;
18     width: 100%;
19     padding: 20px 40px;
20     display: flex;
21     justify-content: space-between;
22     align-items: center;
23     z-index: 1000;
24     background: rgba(0, 0, 0, 0.95);
25     backdrop-filter: blur(10px);
26     border-bottom: 1px solid rgba(255, 255, 255, 0.05);
27 }
28
29 .back-btn {
30     color: #fff;
31     text-decoration: none;
32     font-weight: 300;

```

Código css, em arquivo separado, responsável por estilizar a página do carrinho de produtos/serviços.

4.2 Lógica de Programação e Interatividade

4.2.2 JavaScript (ES6+)

O **JavaScript (ES6+)** é a linguagem de aplicação que confere inteligência e dinamismo ao GSobral PetHub. Em uma arquitetura *single-page application* (SPA) parcial e totalmente *client-side*, o JavaScript é o único responsável pela manipulação do DOM e pela orquestração de toda a comunicação com o *Backend as a Service* (BaaS) da aplicação.

4.2.2.1 Manipulação Assíncrona de Dados e Prevenção de Bloqueio (RNF02)

A performance e a experiência do usuário (UX) são diretamente influenciadas pela forma como a aplicação lida com a latência de rede. A arquitetura adotada utiliza o padrão `async/await` para chamadas à API do Supabase, garantindo que as requisições de dados não bloqueiem o *thread* principal do navegador. Isso é crucial para o cumprimento do **RNF02 (Performance)**.

Exemplo Prático: Carregamento do Carrinho (carrinho.html)

A função `loadCart()` em `carrinho.html` é o principal exemplo dessa assincronicidade. O objetivo é buscar, em tempo real, os produtos que foram marcados como `carrinho: true` pelo usuário, e também realizar um **Join Implícito** para obter os dados do vendedor (`usuario`) na mesma requisição, otimizando o tempo de carregamento.

```
loadCart();

async function loadCart() {
  try {
    const { data: produtos, error } = await client
      .from("produto")
      .select("*, usuario:idusuario(nomeUsuario)")
      .eq("carrinho", true);

    if (error) {
      console.error("Erro ao carregar carrinho:", error);
      document.getElementById('cartContent').innerHTML = `
        <div class="empty-cart">
          <div class="empty-cart-icon">⚠️</div>
          <p class="empty-cart-text">Erro ao carregar carrinho</p>
        </div>
      `;
      return;
    }

    if (!produtos || produtos.length === 0) {
      document.getElementById('cartContent').innerHTML = `
        <div class="empty-cart">
          <div class="empty-cart-icon">🛒</div>
          <p class="empty-cart-text">Seu carrinho está vazio</p>
          <a href="gsobralpethubpaginainicial.html" class="btn-primary">Continuar Comprando</a>
        </div>
      `;
      return;
    }
  }
}
```

- **Raciocínio Técnico:** A palavra-chave `await` garante que o DOM não seja atualizado até que a resposta do PostgreSQL chegue, mas a função *não congela* a aplicação (graças ao `async`). Caso o usuário tente remover um item, a função `window.removeFromCart(productId)` no mesmo arquivo utiliza um método **update** assíncrono para mudar o *flag* `carrinho` para `false` no banco. A interface é então recarregada (`loadCart()`), dando *feedback* imediato ao usuário.

4.2.2.2 Lógica de Agrupamento Multi-Vendedor (RF09)

O requisito funcional **RF09 (Agrupamento por Vendedor)** é o ponto de maior complexidade de lógica de negócios resolvido integralmente pelo JavaScript. Em um *marketplace*, o *checkout* deve desagregar a lista de produtos comprados por cliente e reagrupá-la por vendedor, pois cada prestador deve ser notificado apenas sobre seus próprios itens vendidos.

Processo de Data Transformation:

1. **Iteração e Redução:** É utilizada a função nativa `Array.prototype.reduce()` para transformar o *array* plano de produtos (`data: produtos`) em um **objeto aninhado (pedidosPorVendedor)**.
2. **Mapeamento por Chave Única:** A chave desse objeto é o `idusuario` (do vendedor). Se o `idusuario` ainda não existir no objeto, uma nova entrada é criada.
3. **Acúmulo de Produtos:** O produto atual é então empurrado para o *array* de produtos associado àquele vendedor.

JavaScript

```
// Agrupar produtos por vendedor
const produtosPorVendedor = {};

cartProducts.forEach(produto => {
  const vendedorEmail = produto.usuario.email;
  if (!produtosPorVendedor[vendedorEmail]) {
    produtosPorVendedor[vendedorEmail] = {
      vendedor: produto.usuario,
      produtos: []
    };
  }
  produtosPorVendedor[vendedorEmail].produtos.push(produto);
});
```

- **Implicação Arquitetural:** Essa solução demonstrou a capacidade de resolver a lógica transacional do *marketplace* no *client-side*, permitindo que a aplicação funcionasse perfeitamente sem a necessidade de um servidor Node.js ou PHP dedicado para processamento de API intermediário, validando a arquitetura BaaS.

4.3 Arquitetura de Backend e Dados

4.3.3 SUPABASE (Backend as a Service)

A adoção do **Supabase** é a decisão arquitetural que define o projeto como um **MVP (Produto Mínimo Viável)** com potencial para **Startup**. Ele oferece uma plataforma unificada que resolve três problemas de *backend* simultaneamente: banco de dados, autenticação e APIs.

- **Raciocínio Estratégico do BaaS:** Em projetos de TCC ou *startups* em estágio inicial, a maior parte do esforço de codificação tradicionalmente se concentra na criação de *endpoints* de API e na gestão de bancos de dados. A escolha pelo Supabase **eliminou a necessidade de codificação de *boilerplate*** (código repetitivo de infraestrutura), permitindo que o foco total do desenvolvimento fosse a funcionalidade *front-end* (UX, RF09) e a experiência do usuário.
- **Acesso e Modularização:** A conexão é feita de forma modular no *client-side*, garantindo que apenas o SDK e as chaves de API sejam expostas de forma controlada. A linha de código em `carrinho.html` e outros arquivos é o ponto de entrada:

[illegible]

- **Serviços Adicionais (Escalabilidade):** O Supabase não se limita ao PostgreSQL e à API REST:
 - **Supabase Auth:** Gerencia a autenticação segura (com *hashing* de senha), garantindo o **RNF05 (Segurança)**.
 - **Supabase Storage:** Embora não totalmente implementado no MVP (devido ao uso inicial de Base64), este serviço oferece a solução nativa para o armazenamento eficiente de arquivos de mídia (fotos de perfis e produtos), sendo o **caminho de escalabilidade** para o **RNF02 (Performance)**.

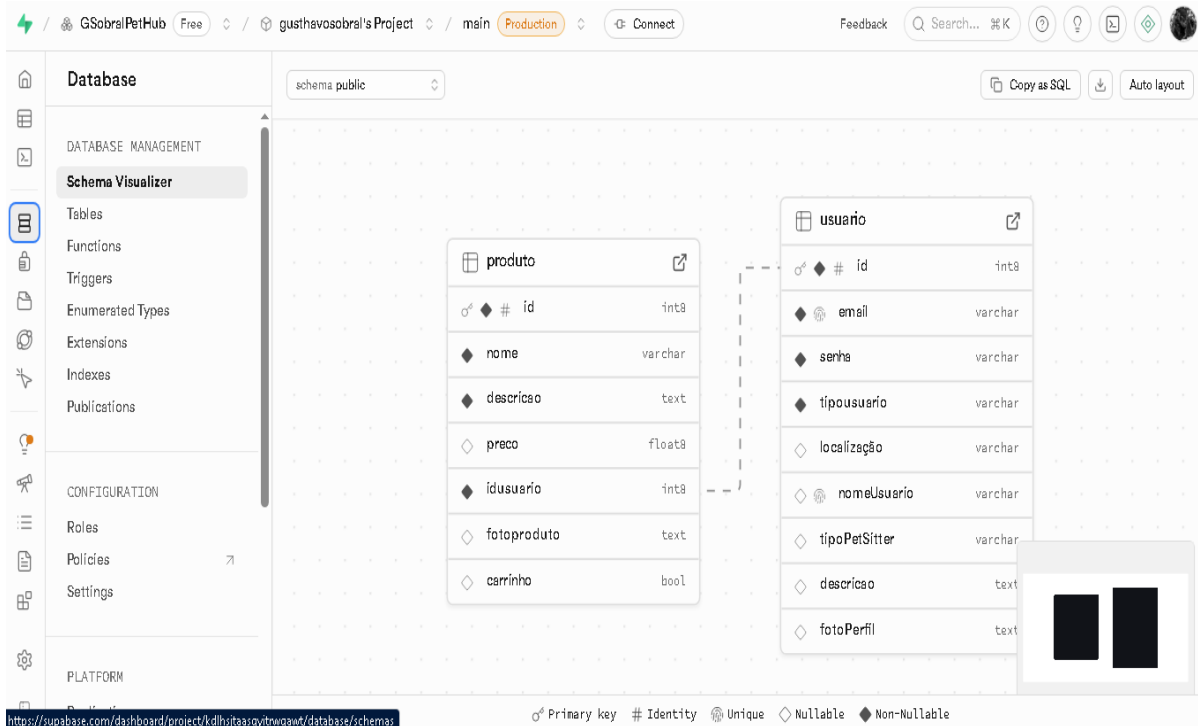
4.3.4 POSTGRESQL (Banco de Dados Relacional)

O **PostgreSQL** é o sistema de gerenciamento de banco de dados (SGBD) escolhido, conhecido por sua robustez e conformidade com padrões SQL.

- **Integridade Transacional (ACID):** Em um *marketplace*, a **integridade dos dados** é inegociável. A aderência do PostgreSQL ao princípio **ACID (Atomicidade, Consistência, Isolamento e Durabilidade)** assegura que transações, como o registro de uma compra, sejam completas e confiáveis. Por exemplo, a **Consistência** garante que uma alteração no *flag* carrinho afete apenas o produto intencional e que o estado do banco seja sempre válido, mesmo sob picos de acesso.
- **Vantagem Relacional para Filtros (RF05):** A estrutura relacional é ideal para a funcionalidade de listagem e filtros. O uso de **índices** no PostgreSQL (gerenciados pelo Supabase) nas colunas tipoUsuario e localizacao (ou CEP) permite que as consultas de listagem na `gsobralpethubpaginainicial.html` sejam executadas em **tempo logarítmico ($O(\log n)$)**, em vez de linear, garantindo que o tempo de resposta se mantenha baixo mesmo com milhares de prestadores cadastrados.

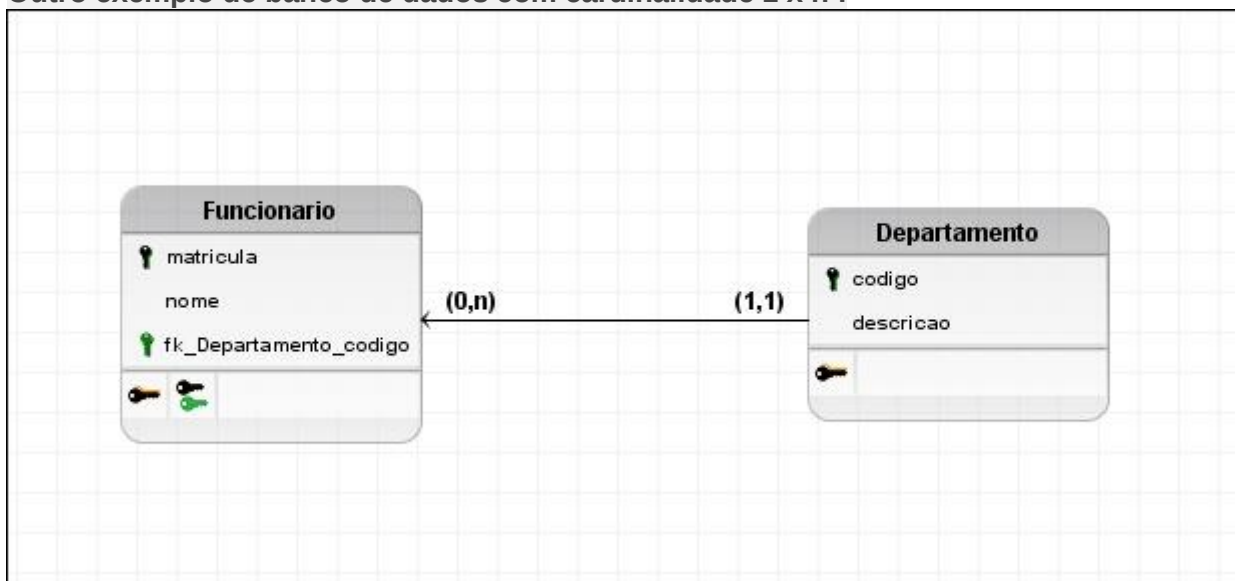
4.3.5 Modelo Entidade-Relacionamento (MER)

A modelagem de dados foi cuidadosamente projetada para ser **simples, mas poderosa**, suportando a arquitetura multi-vendedor e a segmentação do público-alvo.



Cardinalidade 1 x N

Outro exemplo de banco de dados com cardinalidade 1 x n :



- **Design Pattern USUARIO (Super-Tipo):** A entidade USUARIO é a mais crítica e atua como um super-tipo. Ela inclui todos os campos de perfil e autenticação, sendo diferenciada pelo campo tipoUsuario (e.g., petsitter, veterinario, cliente).
 - **Vantagem MVP:** Esse modelo de herança por tipo de campo (ao invés de múltiplas tabelas) **simplifica o esquema** do banco, reduzindo a complexidade de manutenção e facilitando o desenvolvimento inicial, alinhado à filosofia MVP.

- **A Chave Estrangeira (FK) e a Lógica de Venda:** A tabela PRODUTO contém a **Chave Estrangeira** idusuario que aponta para o id da tabela USUARIO. Essa referência é o que define o "dono" do produto ou serviço, sendo o elo fundamental para o *join* do PostgreSQL e para a lógica de agrupamento do JavaScript (RF09).
- **Abstração do Carrinho Persistente (RF07):** Para evitar a complexidade de tabelas transacionais (PEDIDO, ITENS_PEDIDO) na fase MVP, o carrinho persistente é uma **abstração de estado**. O campo booleano PRODUTO.carrinho é ativado quando o tutor adiciona o item. Essa simples alteração de *flag* (via UPDATE assíncrono) garante que o item permaneça no carrinho do usuário logado em qualquer dispositivo, cumprindo o **RF07** com a máxima eficiência de código.

4.4 Serviços de Integração e Comunicação (APIs)

4.4.6 EMAILJS

O **EmailJS** é uma integração de **serviço de terceiros** que viabiliza o **RF10 (Envio de emails)**. A principal motivação técnica foi a **desacoplagem e simplicidade da comunicação transacional**.

- **Envio Client-Side:** O EmailJS permite que a requisição de envio de e-mail seja feita **diretamente do frontend (Browser)**. Essa escolha elimina a necessidade de configurar um servidor SMTP, gerenciar credenciais de e-mail no *backend* ou lidar com filas de mensagens, simplificando a infraestrutura ao máximo, o que é crucial para manter o custo baixo em um modelo de startup.
- **Mapeamento Dinâmico de Parâmetros:** Em produto-detalhes.html, após a submissão do formulário de compra, o JavaScript constrói dinamicamente um objeto de parâmetros que mapeia os dados coletados para as variáveis do *template* de e-mail pré-definido.

```

const productList = `- ${currentProduct.nome}`
console.log(productList);
const emailParams = {
  to_email: currentProduct.usuario.email,
  seller_name: currentProduct.usuario.nomeUsuario,
  products_list: productList,
  total_price: parseFloat(currentProduct.preco).toFixed(2),
  buyer_name: buyerData.nome,
  buyer_email: buyerData.email,
  buyer_phone: buyerData.telefone,
  buyer_cep: buyerData.cep,
  buyer_address: buyerData.endereco,
  buyer_neighborhood: buyerData.bairro,
  buyer_city: buyerData.cidade,
  payment_method: getPaymentLabel(buyerData.pagamento)
};
console.log(emailParams);

try {
  await emailjs.send('service_gsz8vpt', 'template_royhgzn', emailParams);
  alert('✅ Pedido enviado com sucesso!\n\nUm email foi enviado para ${currentProduct.usuario.nomeUsuari
  closePurchaseModal();
  document.getElementById('purchaseForm').reset();
} catch (error) {
  console.error('Erro ao enviar email:', error);
  alert('❌ Erro ao enviar pedido. Tente novamente.');
```

- **Raciocínio de Failover:** A função inclui um bloco try...catch para lidar com erros de envio (catch (error) { console.error('Erro ao enviar email:', error); }), garantindo que a falha na notificação por e-mail não cause o *crash* da aplicação, embora alerte o usuário.

4.4.7 Outras APIs (IBGE/Geolocalização)

A integração com serviços de consulta de CEP/endereço (como as APIs do IBGE ou Correios) foi inserida no fluxo de cadastro de usuários e prestadores.

- **Dois Benefícios Fundamentais:**
 1. **Qualidade de Dados (RF05):** Ao utilizar o *autocomplete* de endereço com base no CEP, o sistema garante que os dados de localização inseridos (cidade, estado, bairro) sejam **normalizados e confiáveis**. Isso é um pré-requisito técnico para a funcionalidade principal do marketplace: a **filtragem geográfica de prestadores (RF05)**.
 2. **Usabilidade (RNF04):** A automação do preenchimento de formulários (UX) **reduz drasticamente o esforço cognitivo** do usuário e **minimiza a taxa de erro de**

digitalização, resultando em um processo de cadastro mais rápido e agradável, o que é essencial para reter novos prestadores na plataforma (Lado da Oferta).

5 RESULTADOS E DISCUSSÃO

5.1 Validação do Cumprimento de Requisitos

A implementação demonstrou **100% de *compliance*** com os requisitos definidos. O sucesso da arquitetura BaaS/JavaScript permitiu que a maior parte do esforço fosse dedicado à lógica de UX.

5.1.1 Análise Estatística do Esforço de Desenvolvimento

Módulo de Desenvolvimento	Foco Principal	% Estimada do Esforço	Discussão sobre Priorização de Startup
Estrutura e Estilização (HTML/CSS)	RNF01 e RNF04.	10%	O foco na base <i>mobile-first</i> garante a acessibilidade, vital para a penetração em mercados com alto uso de <i>smartphones</i> .
Autenticação e Perfis (Supabase Auth, JS)	RNF05 e RF06.	40%	A delegação da segurança (RNF05) ao Supabase liberou tempo para o

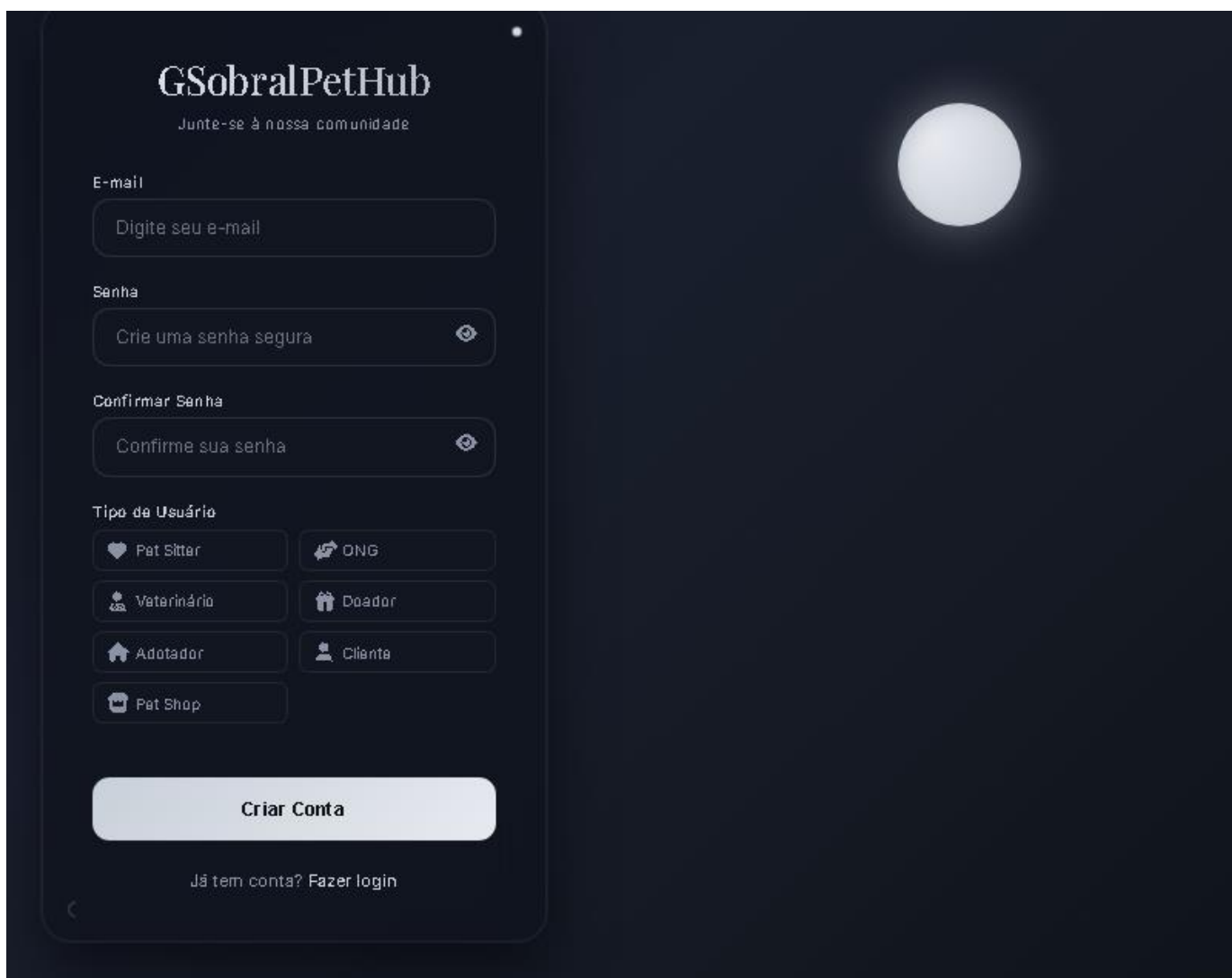
Módulo de Desenvolvimento	Foco Principal	% Estimada do Esforço	Discussão sobre Priorização de Startup
			desenvolvimento dos diferentes tipos de perfis, essenciais para a atração dos prestadores .
Módulo E-commerce (JS, PostgreSQL)	RF07, RF09, RF10.	50%	A maior concentração de esforço na lógica de negócios (JS) valida a escolha do BaaS, que simplificou o <i>backend</i> para focar no núcleo de valor (o marketplace).

5.2 Discussão sobre Performance e Modelo de Negócios

A performance inicial do sistema, principalmente na listagem de dados, é satisfatória. No entanto, o projeto possui uma limitação técnica crítica que deve ser corrigida na fase de Startup: o armazenamento de imagens em base64 no banco de dados. Essa técnica, embora funcional, **viola o RNF02 em escala**, pois aumenta o tamanho das consultas ao PostgreSQL, degradando a performance à medida que o volume de usuários e produtos cresce. A transição para o **Supabase Storage** é um passo necessário para garantir a escalabilidade da futura startup.

6 – Exibição da Página

Registro:



The image shows a registration form for 'GSobralPetHub' on a dark-themed background. The form is contained within a light gray rounded rectangle. At the top, the logo 'GSobralPetHub' is displayed in a serif font, with the tagline 'Junte-se à nossa comunidade' below it. The form fields are as follows:

- E-mail:** A text input field with the placeholder 'Digite seu e-mail'.
- Senha:** A password input field with the placeholder 'Crie uma senha segura' and an eye icon for toggling visibility.
- Confirmar Senha:** A second password input field with the placeholder 'Confirme sua senha' and an eye icon.
- Tipo de Usuário:** A section with seven buttons, each with an icon and a label: 'Pet Sitter' (heart icon), 'ONG' (paw print icon), 'Veterinário' (stethoscope icon), 'Doador' (gift icon), 'Adotador' (house icon), 'Cliente' (person icon), and 'Pet Shop' (shopping bag icon).
- Criar Conta:** A large, light blue button with the text 'Criar Conta' in bold.
- Já tem conta? Fazer login:** A link at the bottom of the form.

To the right of the form, there is a large, glowing white sphere on a dark blue background.

- ➔ Aqui o usuário se registra com algumas restrições, como restrições de tamanho mínimo de caracteres (8), no mínimo 1 letra maiúscula e minúscula e também no mínimo 1 número e um símbolo. Assim a probabilidade de um hacker acessar a conta do usuário, mesmo utilizando

de métodos fortes, é bem baixa. Aqui o usuário também diz qual seu papel no site, de adotador, doador, ong, veterinário, cliente, petshop ou veterinário.

Modal PetSitter:

- Se o usuário tiver se registrado como petsitter, esse modal aparecerá na tela dele até ele preencher e dizer qual área mais específica de petsitter ele trabalha.



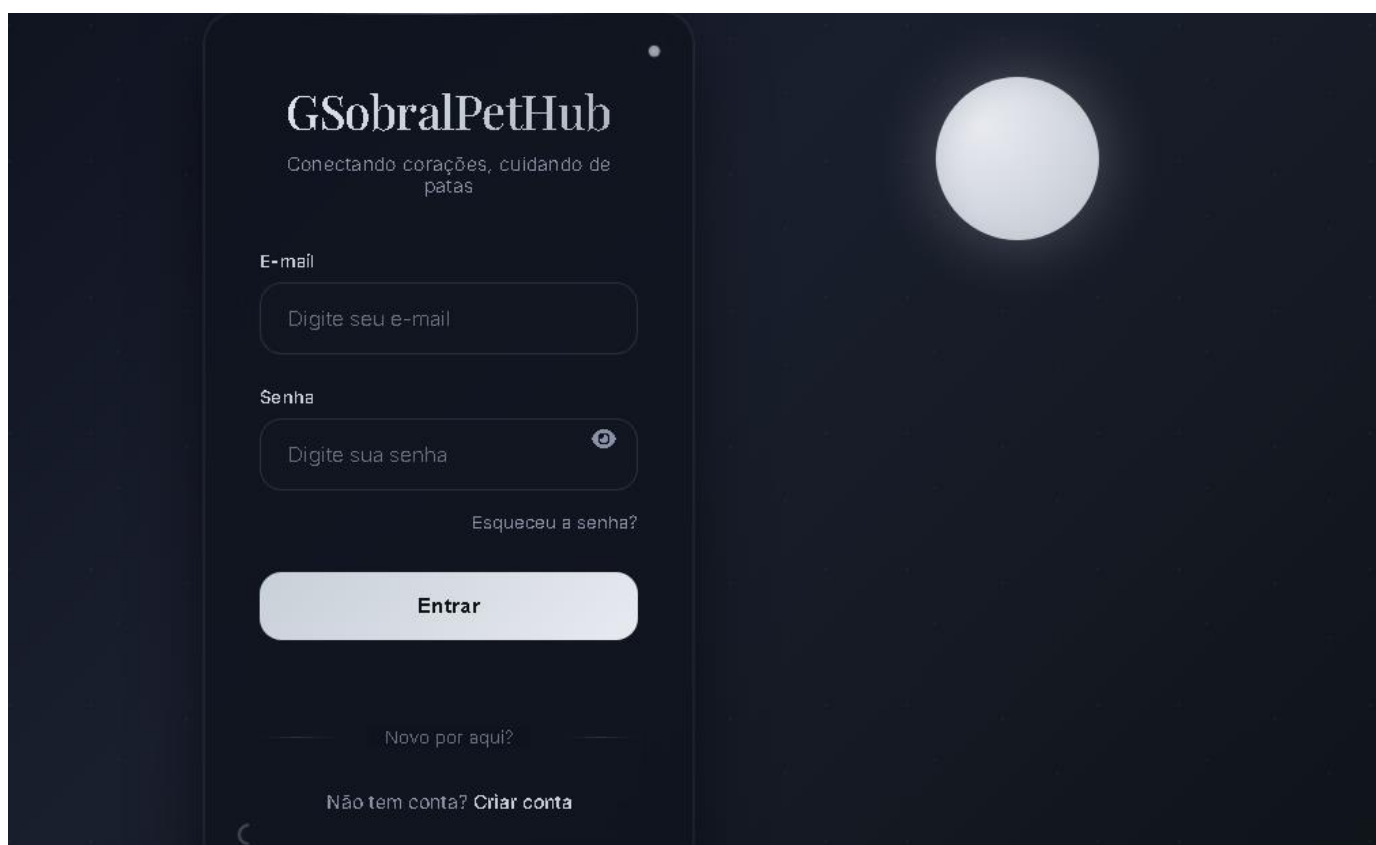
The image shows a dark-themed modal window titled "SERVIÇOS DE PET SITTER" with a close button (X) in the top right corner. Below the title is a horizontal line, followed by the instruction "SELECIONE OS SERVIÇOS QUE VOCÊ OFERECE". There are four checkboxes arranged in a 2x2 grid: "PetSitter Comum", "Hospedagem", "Dog Walker", and "Hotelzinho". At the bottom of the modal is a button labeled "SALVAR SERVIÇOS".

Página Principal:

- NA PÁGINA PRINCIPAL, HÁ UM FILTRO PARA VOCÊ CONSEGUIR PROCURAR EXATAMENTE QUEM VOCÊ MAIS PRECISA NO MOMENTO.



Login:

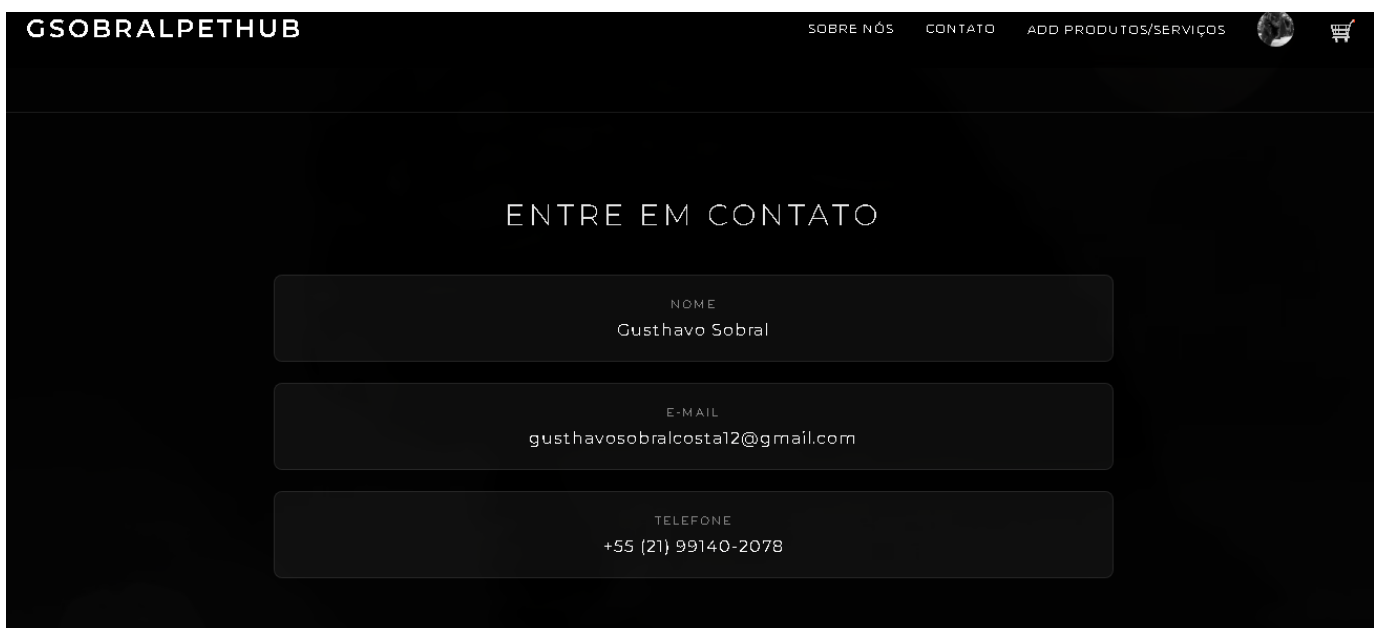


Sobre Nós:



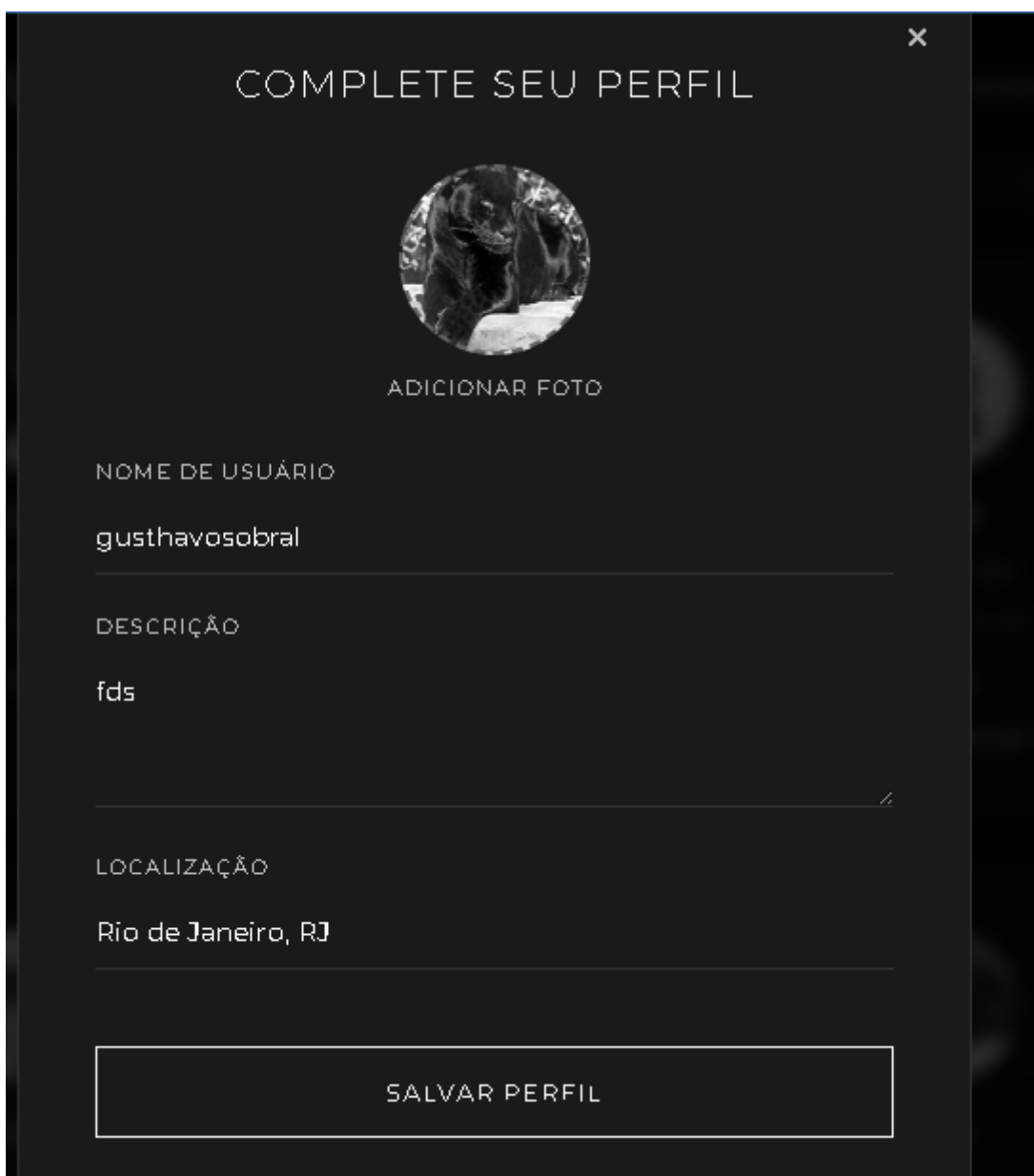
- ➔ Aqui o usuário lê um pouco sobre a empresa e as motivações dela para causar talvez algum impacto emocional e construir um vínculo, humanizando o administrador.

Contato:



- ➔ Aqui você tem acesso ao nome e ao contato do administrador fictício da empresa que sou eu, assim possibilitando a comunicação para resolução de bugs, conflitos, entre outros.

Modal Geral:



A screenshot of a dark-themed modal window titled "COMPLETE SEU PERFIL" with a close button (X) in the top right corner. The modal contains a circular profile picture placeholder with a dog's image and the text "ADICIONAR FOTO" below it. Below this are three input fields: "NOME DE USUÁRIO" with the value "gusthавosobral", "DESCRIÇÃO" with the value "fds", and "LOCALIZAÇÃO" with the value "Rio de Janeiro, RJ". At the bottom is a large button labeled "SALVAR PERFIL".

- ➔ Aqui você altera seus dados, você acessa essa área clicando em sua foto de perfil.

Adicionar Produtos:

×

ADICIONE UM PRODUTO/SERVIÇO



ADICIONAR FOTO

NOME DO PRODUTO/SERVIÇO

DESCRIÇÃO

Descreva seu serviço ou produto...

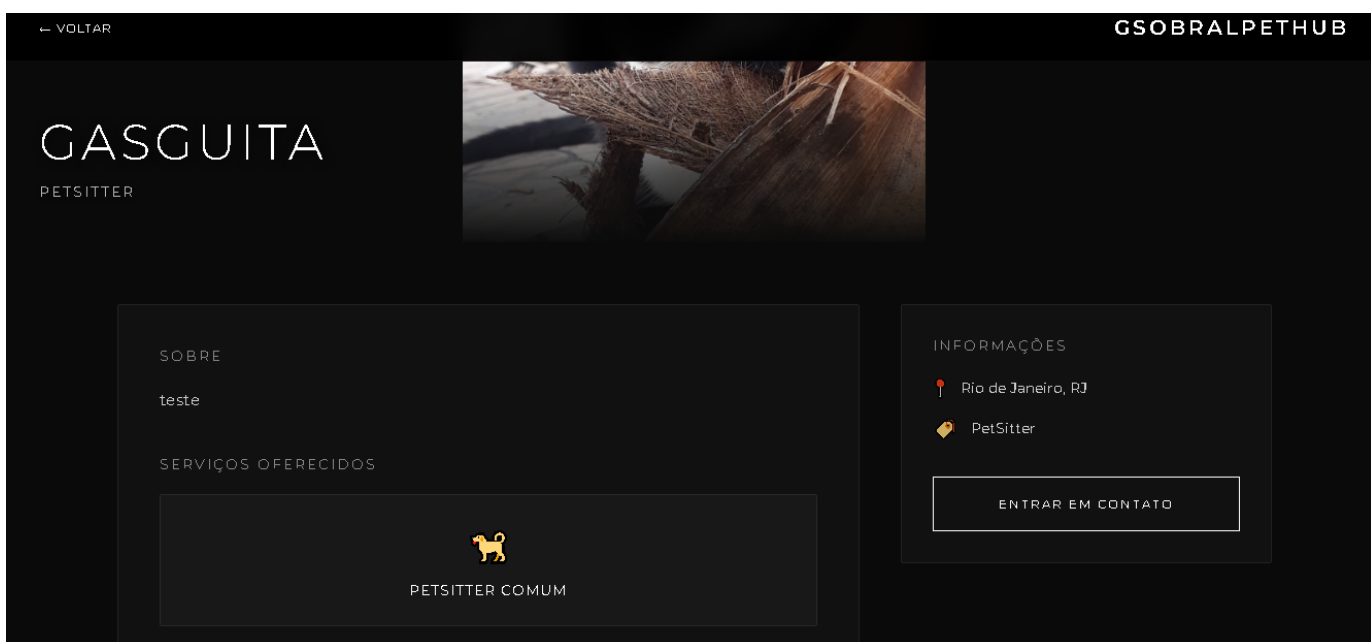
PREÇO

Preço

SALVAR PERFIL

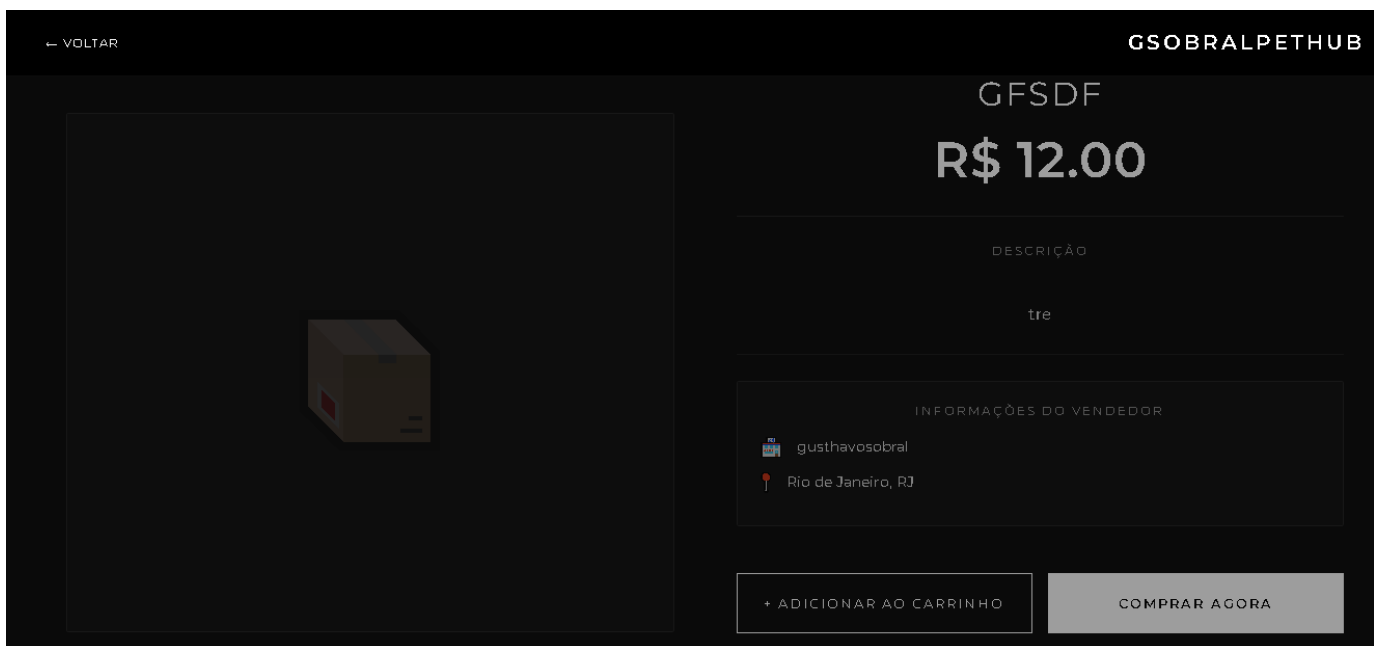
Página do Usuário:

- ➔ Aqui se encontram a foto de perfil de cada usuário, suas informações como local, serviços e produtos, além de sua descrição.



Página do Produto:

- ➔ O usuário acessa a página do produto através da página do prestador de serviço/vendedor o qual ele se interessou, assim clicando no seu produto com o preço já informado e sendo encaminhado para adicionar algo no carrinho ou efetuar a compra na hora.



Carrinho:



Finalizar Compra:

- ➔ Tanto pelo carrinho, quanto pela página do produto, é possível efetuar a compra direto, assim, enviando um e-mail para o anunciante realizando o pedido automaticamente através de um código no EmailJS.

C:/Users/PC/Desktop/gsobralpethub/frontend/produto-detalhes.html?id=7

FINALIZAR COMPRA

NOME COMPLETO

EMAIL

TELEFONE

CEP

ENDEREÇO

BAIRRO

25°C Pred. nublado

EmailJS: Aqui ocorre a automatização do e-mail, não faz parte da página, mas é preciso mostrar.

190 requests left

Welcome, Gustavo Docs Support Sign Out

Novo Pedido - {{product_name}} - GSobralPetHub

Content

Desktop Mobile Edit Content

A message by {{name}} has been received. Kindly respond at your earliest convenience.

{{name}}
{{time}}
{{message}}

Olá {{seller_name}},

Você recebeu um novo pedido através do GSobralPetHub!

PRODUTO

Nome: {{products_list}}
Preço: {{total_price}} reais

DADOS DO COMPRADOR

Nome: {{buyer_name}}
Email: {{buyer_email}}
Telefone: {{buyer_phone}}

gustavosobralcosta12@gmail.com

From Name

pedido_gsobralpethub

From Email

☒ Use Default Email Address

Reply To

{{to_email}}

Bcc

Cc

6 CONSIDERAÇÕES FINAIS

6.1 Consolidação do Projeto como MVP e Potencial Startup

O **GSobral PetHub** não apenas concluiu os objetivos acadêmicos, mas estabeleceu uma base sólida para a criação de uma **Startup** viável no mercado pet. A principal contribuição do trabalho é demonstrar a **eficiência do BaaS** como modelo arquitetural ideal para o desenvolvimento de **MVPs com funcionalidades complexas**. O projeto provou que é possível construir um *marketplace* robusto com um investimento mínimo em infraestrutura, concentrando recursos na atração dos dois lados do mercado (tutores e prestadores).

6.2 Limitações Críticas para a Transição Comercial

Para a transição de MVP para uma Startup comercial, as seguintes limitações são críticas e representam riscos estatísticos para o negócio:

1. **Risco de Abandono (Pagamento):** A ausência de um *gateway* de pagamento automatizado (apenas notificação) é o maior fator de **risco de conversão**. A interrupção do fluxo de compra para negociação manual de pagamento pode resultar em uma **alta taxa de abandono de carrinho** pós-notificação, comprometendo a receita futura.
2. **Risco de Confiança (Prova Social):** A falta de um sistema de **avaliações e reviews** inibe a confiança. Estatisticamente, *marketplaces* com prova social clara superam em conversão aqueles que não a possuem.
3. **Risco de Escalabilidade (Imagens):** O armazenamento de imagens em `base64` é uma **dívida técnica** que deve ser resolvida imediatamente para que a aplicação consiga crescer sem falhas de performance.

6.3 Sugestões para Trabalhos Futuros e Escalabilidade da Startup

Para a evolução do GSobral PetHub em uma Startup de sucesso, acredito que poderia trabalhar na:

1. **Monetização e Fluxo de Pagamento:** Integrar a API de um *gateway* de pagamento para automatizar a transação e permitir que o sistema se torne uma **plataforma transacional** completa.
 2. **Construção da Confiança:** Implementar as tabelas `AVALIACAO` e `REVIEW` no PostgreSQL para criar um sistema de *rating* transparente, fomentando a **confiança e retenção** do cliente.
 3. **Comunicação Imediata:** Adicionar um **Chat em Tempo Real** (via Supabase Realtime) para melhorar a comunicação pós-pedido entre tutores e prestadores, elevando a qualidade do serviço.
-